

The idea is to compile code written in the synchronous blocking model in C to an asynchronous model underneath.

Prereqs:

How FilC tracks provenance (origin) of pointers: <https://fil-c.org/how>

The Gist

```
int *p = malloc(20);
// filc emits some LLVM IR to track this pointer as it is being passed
// around so that it can bounds-check it.
// what it stores a tag: (start_address | end_address)
// the buffer is valid from start_address to end_address
// filc uses this for bounds checking when `p` is derefed

p[21] // this panics at runtime in filc
p[0] // passes the runtime bounds check
```

This **"tag"** propogation that FilC compiler already does will be used by us to *lazily* evaluate results at runtime. We will get to how and when we do this later.

Obviously this adds a bit of overhead whenever a `load` from a pointer deref is done. FilC cites about 1.5x to 2x speed decrease from gcc. Note however that this is based on the current FilC release which is still in its infancy.

Complicated programs can also work using this provenance tracking since the provenance is propagated:

```
int* p = (int*)malloc(20);
int* d = p + 2; // provenance of `d` is calculated at runtime and appended
to it
// provenance tag of d = (start_addr + 2 , end_addr)
```

The advantage of `iouring` that we want to abuse:

```
// say we make 3 independent syscalls
syscallA();
syscallB();
syscallC();
// each of these syscalls block and RUN ON THE SAME THREAD AS THE CURRENT
```

PROCESS in the traditional synchronous model on linux

// what we will do is : make syscalls lazily evaluate in the background and allow it to be executed on multiple cores (as a virtue of io-uring)

The General Idea:

// say we want this function to possibly run in async

```
#pragma async
```

```
int* balls(int* mutable_thingy, const int immutable_thingy) {  
}
```

```
int main() {  
    int a = 2;  
    int b = 2;
```

// now that we do a call to balls, we submit a request in some queue or to some asynchronous model (in the case of syscalls this will be io-uring)

```
int* result = balls(&a, b);
```

// we emit IR to attach provenance tags to result, a

// provenance tag is not emitted for `b` since it cannot be modified

printf("%d", a); // now that a is read or used here, we will now check if the result of the function call returned. If the function call returned then we have the result at the cost of NO CONTEXT SWITCHES IF BALLS WAS A SYSCALL.

// If the result hasn't arrived yet then we spinwait on it.

```
printf("%d", b); // do the same as above for b  
}
```

Looking at some cases:

Dependent Syscalls

```
int main() {  
    int a = 2;  
    int b = 3;  
  
    int result = balls(&a, b);  
    // attached provenance tags to a, `result`
```

```

    // this async function call requires the result of the first async call.
    // now we have no problem as, when we pass `result` to another function
    we trigger the procedure for checking if result has arrived just like before
    another_async_function(result, b);
}

```

Dependent Syscalls (But the Dependence is Hidden and not apparent in the code)

```

int main() {
    int a = 2;
    int b = 3;

    int result = balls(&a , b);
    // attached provenance tags to a, `result`

    int c = 3;
    // say the result of this syscall is DEPENDENT ON THE FACT THAT THE
    FIRST FUNCTION CALL RETURNED
    another_async_function(&c, b);
    // in this case the user must explicitly specify to either fall back to
    synchrnous execution or must annotate that the second function call is
    dependent on the first system call.

    // FOR HANDLING SUCH CASES WE DO A CLEVER TRICK (atleast i think its
    clever):
    // EVERY FUNCTION TAKES A HIDDEN LAST ARGUMENT FOR DEPENDENCE TRACKING:
    THIS USES THE PREVIOUS MECHANISM OF WAITING FOR A FUNCTION ARGUMENT TO BE
    EVALUATED

    // signature written by the programmer:
    int balls(int* a, int b);
    // signature actually generated
    int balls(int* a, int b, void* PROVENANCE = 0)
    // 0 or NULL means that this is prallely executed according to above
    scheme.

    // now lets say that there are 2 depdenent syscalls. THE USER IS
    REQUIRED TO EXPLICITELY DECLARE IT USING THE FOLLOWING

    void* PROVTRACKER = prov_alloc(); // this function needs to be
    implemented by us

```

```

int a = 2;
int b = 3;
balls(&a, b, PROVTRACKER);
// provenance tracker is added to PROVTRACKER according to previous
scheme

int c = 3;
int d = 4;
balls(&c, d, PROVTRACKER);
// NOW THAT THE USER HAS DECLARED that the above 2 calls must be one-
after-the-other, since PROVTRACKER is passed as an argument, this call must
wait for the result of the first syscall.
}

```

Now that we described this generic asynchronous scheme, we are going to implement this for syscall functions that use iouring as the asynchronous backend.

Future Scope I See

- A generic asynchronous backend interface
- Users can port different parallel execution models for the asynchronous backend or reserve different cores for certain functions
- HOPEFULLY SHOULD HAVE BETTER PERFORMANCE